

Top 10 Key Takeaways

Chunks 1–9 · 14:03–15:26 · Instructors Marlon Kuzmick (MK) and Madeleine Woods (MW), with Adam on the repo tour and Becca on Git

Distilled from the diarized session transcript at [transcripts/20260609-transcript.md](#). Conceptual Git explainer from Becca, the Learning Lab's executive director; faculty projects from Jordan, Jungyoon, and Lindsay.

- Day 2 is the bridge — from chat and Cowork into Claude Code, and the desktop app's Code panel is the democratic on-ramp.** "Today we are finally crossing the bridge from chat and Cowork into Code" — flick to the third panel on the right. You can do "ninety-nine percent of everything Claude can do" in the desktop app's Code tab, so the whole room stays there; developers who want the CLI / VS Code (IDE) path set it up after class (~30 min). New names for one idea: a new *chat* (Chat), a new *task* (Cowork), a new *session* (Code).
- Three ways to get the repo onto your machine — the easiest is to just ask Claude to clone it.** Download the green **Code** → **zip** and unzip it; run `git clone <url>` in the terminal; or, in the desktop app, create a folder, select it, and say "*can you clone this repo here?*", then paste the URL and hit allow. One catch: copy the URL from the green **Code** button's clipboard icon, not the plain emailed link — "it's not smart enough to change the syntax" otherwise.
- One dedicated Development folder — never your desktop or root drive.** The Day-1 discipline carries into Code: make one folder you'll remember, clone into it, and point Claude there. "I wouldn't give it access to my desktop or, you know, my root drive." Cowork and Code ask permission per file, so a contained folder keeps that manageable.
- Becca's conceptual Git: Google Docs version history, but for a whole project.** Git was built by coders to track collaborative versions of code — but it's "a very general system for keeping track of collaborative work that has multiple versions." The analogy: how useful Google Docs version history is, except for an entire folder of files, tracking every change *and* the human comments about *why* a batch of changes was made.
- Three reasons Git matters even if you never write code.** (1) **History** — pull something back out of your project's past. (2) **Claude reads it first** — "Claude is built to look very first thing at the Git repository to understand the history of the project," getting around the limited context window across sessions. (3) **Safety net** — when AI changes a file in a direction you didn't intend, a tracked repo makes it "extremely easy to go back to the earlier versions and pull those back out."
- How Git works: a hidden .git folder, snapshots, and a remote on GitHub.** Editing does nothing until you decide "*now's the moment to take a picture*" — choose the changes, add a message saying what they mean, and **commit**. GitHub is "a remote repository out on the internet which stores a copy of what's locally on your machine" — private or public — a second layer of protection and the way you share. Even if you deleted your local `.git`, the GitHub copy survives.
- .gitignore keeps secrets out — never push API keys or passwords.** Some files (an API key, a password, proprietary or very large files) are "really only for you." Don't put them in your chats with Claude, and definitely don't push them to GitHub. A special `.gitignore` file tells Git never to include them — and you can just ask Claude to make one.
- You don't learn Git commands — Claude is fluent, and "Create PR" basically means commit.** Claude Code surfaces buttons (create a repo, commit, push), or you ask in plain language. The one bit of jargon: it offers to "**create a pull request**," which "essentially means create it, commit it, and pull it onto the main branch." The on-screen counter of additions/subtractions since your last commit is a nudge — **commit often**; each commit is a more granular memory and a point to jump back to.
- Even solo, you're now doing collaborative work — agents commit on your behalf.** Ask Claude to "summarise the work people did on this branch today" and it shows every commit down to the minute, who made it, and which branches merged in. Picture a **tree**: a contributor's work arrives as a **branch**, a pull request "knocks on the door of the main repository," and you open the door, review, and merge — so many people (or many Claude agents) work without overwriting each other. Demo: a first-time contributor's spelling-fix PR merged in, then `git pull` ("hey Claude, there was a change on GitHub, can you pull?").
- Point Claude at one file by its path — and remember it's all just text, "turtles all the way down."** The core move: click the **three dots** next to a file → **copy path** → paste it into your prompt ("the unique path to that resource on your computer," like a URL). Students could query a transcript you've vetted — "*top five takeaways from Day 1*" — and ask their own questions. In the Code tab you can also **edit files yourself** (unlike chat/Cowork artifacts you must ask to change) — what you want for "intellectual work you care about." Handing over a specific path is far cheaper than making Claude search the whole repo against your 200K–1M-token window.

SECONDARY POINTS WORTH KEEPING

- The Homebrew tax for some Macs.** Git was missing on some older Macs; install **Homebrew** (`brew.sh`), run the two lines it prints, then `brew install git`. In Terminal your password types invisibly — "use the force and just keep typing."
- Find your files in the Code tab.** The views icon (top-right) only appears *after* you ask a question; click it → **Files** to see the whole repo. VS Code shows files by default; in Claude Code you nudge it.
- .docx / .xlsx are proprietary, not text.** Claude reads them in and writes them out via skills, but works best with **simple text / markdown**. A folder of PDFs or `.docx` files can be batch-converted.
- Recipes are remixable starting points.** Copy a recipe's path and say "*make me a folder using this scaffolding in faculty-recipes for [my course], and search the internet for starter papers.*" Shown live: gradebook visualization, a video word-frequency search (the Shia LaBeouf "just do it" corpus, built in ~9 min), a reworked Math course, a pared-down bioengineering course pack.
- Keep big stuff local, share small stuff.** A full Canvas download can be worked on locally but needn't be pushed to GitHub — the shared course packs were kept deliberately small.
- HTML opens in the browser.** Any `.html` file in the repo can be viewed in Claude or by pasting its path into Chrome/Edge; later days cover publishing it for students.
- Tomorrow (Day 3):** the file types higher in the context hierarchy that load automatically — **CLAUDE.md** and **skills**.